

Module 4: Decision Making in PL/SQL (Conditional Statements)

Duration

6–8 Hours (Theory + Hands-on Laboratory)

Learning Outcomes

After completing this module, students will be able to:

- Understand the concept of decision making.
 - Write programs using the IF statement.
 - Use IF-ELSE to implement two-way decisions.
 - Implement Nested IF statements.
 - Write multiple conditions using the ELSIF ladder.
 - Use the CASE statement effectively.
 - Compare IF and CASE statements.
 - Develop real-world decision-making applications.
-

4.1 Introduction to Decision Making

What is Decision Making?

Decision making is the process of executing a particular block of code based on whether a specified condition evaluates to **TRUE** or **FALSE**.

In programming, many situations require the program to make decisions before performing an action.

For example:

- Should a student be declared pass or fail?
- Is an employee eligible for a bonus?
- Can a customer receive a discount?
- Is a voter eligible to vote?
- Is a user authenticated?

These decisions are made using **conditional statements**.

Real-Life Example

Suppose a student secures **38 marks**.

Question:

Should the student pass?

Rule:

If Marks ≥ 40

Student Pass

Else

Student Fail

Similarly,

If Salary > 50000

Bonus

Else

No Bonus

Every software application follows this logic.

Types of Conditional Statements

PL/SQL provides five decision-making statements.

Statement	Purpose
IF	Single condition
IF...ELSE	Two-way decision
Nested IF	Multiple IF inside IF
ELSIF Ladder	Multiple conditions
CASE Statement	Alternative to ELSIF

4.2 IF Statement

What is IF Statement?

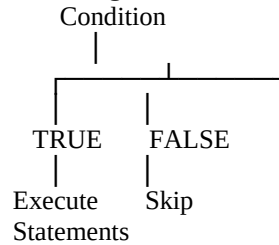
The **IF statement** executes a block of code **only when the specified condition is TRUE**.

If the condition evaluates to FALSE, the program skips the block.

Syntax

```
IF condition THEN
    statements;
END IF;
```

Flow Diagram



Example 1: Voting Eligibility

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    age NUMBER := 20;
```

```
BEGIN
```

```
    IF age >= 18 THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Eligible to Vote');
```

```
    END IF;
```

```
END;
```

```
/
```

Output

Eligible to Vote

Example 2: Student Pass

```
DECLARE
```

```
    marks NUMBER := 75;
```

```
BEGIN
```

```
    IF marks >= 40 THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Student Passed');
```

```
    END IF;
```

```
END;
```

```
/
```

Example 3: Positive Number

```
DECLARE
```

```
    num NUMBER := 15;
```

```
BEGIN
```

```
    IF num > 0 THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Positive Number');
```

```
    END IF;
```

```
END;  
/
```

Characteristics of IF Statement

- Only one condition
- Executes only when TRUE
- No FALSE block

4.3 IF...ELSE Statement

Why IF...ELSE?

Sometimes we need to execute one block when the condition is TRUE and another block when it is FALSE.

Example

If Rain

Take Umbrella

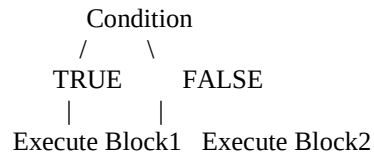
Else

Wear Sunglasses

Syntax

```
IF condition THEN  
    statements;  
ELSE  
    statements;  
END IF;
```

Flowchart



Example 1: Pass or Fail

```
DECLARE  
    marks NUMBER := 35;  
BEGIN  
  
    IF marks >= 40 THEN  
        DBMS_OUTPUT.PUT_LINE('PASS');  
    ELSE  
        DBMS_OUTPUT.PUT_LINE('FAIL');  
    END IF;
```

```
END;  
/
```

Output

FAIL

Example 2: Even or Odd

```
DECLARE  
    num NUMBER := 25;  
BEGIN  
  
    IF MOD(num,2)=0 THEN  
        DBMS_OUTPUT.PUT_LINE('Even');  
    ELSE
```

```

        DBMS_OUTPUT.PUT_LINE('Odd');
    END IF;

END;
/

```

Example 3: Largest of Two Numbers

```

DECLARE
    a NUMBER:=50;
    b NUMBER:=75;
BEGIN

    IF a>b THEN
        DBMS_OUTPUT.PUT_LINE('Largest = '||a);
    ELSE
        DBMS_OUTPUT.PUT_LINE('Largest = '||b);
    END IF;

END;
/

```

4.4 Nested IF Statement

What is Nested IF?

A Nested IF is an IF statement inside another IF statement.
Used when one decision depends on another.

Example

Admission Process
If Marks >= 60

If Interview Passed

Admission Confirmed

Else

Interview Failed

Else

Not Eligible

Syntax

```

IF condition1 THEN

    IF condition2 THEN

    END IF;

END IF;

```

Example

```

DECLARE
    marks NUMBER:=80;
    interview CHAR(1):='Y';

```

```

BEGIN

    IF marks >= 60 THEN

        IF interview = 'Y' THEN

            DBMS_OUTPUT.PUT_LINE('Admission Confirmed');

        ELSE

            DBMS_OUTPUT.PUT_LINE('Interview Failed');

        END IF;

    ELSE

        DBMS_OUTPUT.PUT_LINE('Not Eligible');

    END IF;

END;
/

```

4.5 ELSIF Ladder

Why ELSIF?

Suppose we need multiple conditions.

Student Grades

Marks $\geq 90 \rightarrow A+$

Marks $\geq 80 \rightarrow A$

Marks $\geq 70 \rightarrow B$

Marks $\geq 60 \rightarrow C$

Else $\rightarrow$ Fail

Writing multiple IF statements is inefficient.

The ELSIF ladder provides a cleaner solution.

Syntax

```
IF condition THEN
```

```
ELSEIF condition THEN
```

```
ELSE
```

```
END IF;
```

Note: In Oracle PL/SQL, the correct keyword is **ELSIF** (not ELSEIF).

```
IF condition1 THEN
```

```
    statements;
```

```
ELSIF condition2 THEN
```

```
    statements;
```

```
ELSIF condition3 THEN
```

```
    statements;
```

```
ELSE
```

```
    statements;
```

END IF;

Example: Grade Calculator

DECLARE

marks NUMBER:=82;

BEGIN

IF marks>=90 THEN

DBMS_OUTPUT.PUT_LINE('Grade A+');

ELSIF marks>=80 THEN

DBMS_OUTPUT.PUT_LINE('Grade A');

ELSIF marks>=70 THEN

DBMS_OUTPUT.PUT_LINE('Grade B');

ELSIF marks>=60 THEN

DBMS_OUTPUT.PUT_LINE('Grade C');

ELSE

DBMS_OUTPUT.PUT_LINE('Fail');

END IF;

END;

/

Output

Grade A

Example: Income Tax Slab

DECLARE

income NUMBER:=650000;

BEGIN

IF income<=250000 THEN

DBMS_OUTPUT.PUT_LINE('No Tax');

ELSIF income<=500000 THEN

DBMS_OUTPUT.PUT_LINE('5% Tax');

ELSIF income<=1000000 THEN

DBMS_OUTPUT.PUT_LINE('20% Tax');

ELSE

DBMS_OUTPUT.PUT_LINE('30% Tax');

```
END IF;  
  
END;  
/
```

4.6 CASE Statement

What is CASE?

The CASE statement is an alternative to multiple ELSIF statements. It improves readability when checking many discrete values.

Types of CASE

1. Simple CASE

CASE expression

```
WHEN value1 THEN
```

```
WHEN value2 THEN
```

```
ELSE
```

```
END CASE;
```

Example

```
DECLARE  
    day_number NUMBER:=3;  
BEGIN  
  
    CASE day_number  
  
        WHEN 1 THEN  
            DBMS_OUTPUT.PUT_LINE('Monday');  
  
        WHEN 2 THEN  
            DBMS_OUTPUT.PUT_LINE('Tuesday');  
  
        WHEN 3 THEN  
            DBMS_OUTPUT.PUT_LINE('Wednesday');  
  
        ELSE  
            DBMS_OUTPUT.PUT_LINE('Invalid Day');  
  
    END CASE;  
  
END;  
/
```

2. Searched CASE

CASE

```
WHEN condition THEN
```

```
WHEN condition THEN
```

```
ELSE
```

```
END CASE;
```

Example

```
DECLARE
    marks NUMBER:=72;
BEGIN

    CASE

        WHEN marks>=90 THEN
            DBMS_OUTPUT.PUT_LINE('Grade A+');

        WHEN marks>=80 THEN
            DBMS_OUTPUT.PUT_LINE('Grade A');

        WHEN marks>=70 THEN
            DBMS_OUTPUT.PUT_LINE('Grade B');

        ELSE
            DBMS_OUTPUT.PUT_LINE('Need Improvement');

    END CASE;

END;
/
```

IF vs CASE**IF Statement**

Best for complex logical conditions
Easier for range-based conditions
Supports nested conditions
Slightly longer

CASE Statement

Best for multiple discrete values
Cleaner for menu or code selection
More readable for many alternatives
More concise

Real-World Applications

Decision-making statements are used in:

- ATM Transaction Validation
- Banking Loan Approval
- Payroll Bonus Calculation
- Income Tax Computation
- Student Result Processing
- Hospital Patient Categorization
- E-Commerce Discount Engine
- Employee Promotion Eligibility
- University Admission Systems
- Attendance Management

Complete Program: Employee Bonus

```
DECLARE
    salary NUMBER:=45000;
    bonus NUMBER:=0;

BEGIN

    IF salary>=50000 THEN
        bonus:=salary*0.20;
```



```
ELSIF salary >= 30000 THEN
    bonus := salary * 0.10;

ELSE
    bonus := salary * 0.05;

END IF;

DBMS_OUTPUT.PUT_LINE('Bonus = ' || bonus);

END;
/
```

Common Errors by Beginners

- Forgetting END IF;
- Using = instead of := for assignments.
- Writing ELSEIF instead of ELSIF.
- Missing THEN after the condition.
- Using CASE when range-based IF logic is more suitable.
- Not enabling SET SERVEROUTPUT ON; when using DBMS_OUTPUT.PUT_LINE.

Summary

In this module, we introduced **decision-making constructs** in PL/SQL. Students learned how to use IF, IF...ELSE, Nested IF, ELSIF, and CASE statements to control the flow of execution based on conditions. These constructs enable developers to implement business rules such as grading systems, bonus calculations, eligibility checks, and approval workflows.

Laboratory Exercises

1. Check whether a student has passed or failed.
2. Determine whether a number is even or odd.
3. Find the largest of two numbers.
4. Find the largest of three numbers using Nested IF.
5. Determine voting eligibility based on age.
6. Calculate grades using the ELSIF ladder.
7. Develop an income tax calculator.
8. Display the day of the week using a CASE statement.
9. Calculate employee bonus based on salary slabs.
10. Create a menu-driven calculator using a CASE statement.

Viva/Interview Questions

1. What is decision making in PL/SQL?
2. Differentiate between IF and IF...ELSE.
3. What is a Nested IF statement?
4. Explain the purpose of the ELSIF ladder.
5. Why does PL/SQL use ELSIF instead of ELSEIF?
6. What is the difference between a Simple CASE and a Searched CASE?
7. When would you choose CASE over IF?
8. Can multiple ELSIF blocks be used in a single IF statement?
9. What are common mistakes when writing conditional statements?
10. Give two real-world applications of decision-making statements in PL/SQL.